

1 From Bit Commitment and ZKP to Secure Computation

In the previous lectures, we studied cryptographic mechanisms such as bit commitment and zero-knowledge proofs. A bit commitment allows a party to commit to a value while keeping it hidden, with the commitment preventing the party from changing the value later. A zero-knowledge proof allows a party to prove the validity of a statement, or prove knowledge of a secret, without revealing the secret itself.

These primitives illustrate an important principle in cryptography: parties can interact and establish properties about private information without necessarily revealing that information. This naturally leads to a broader question:

Can multiple parties jointly perform a computation using their private data without revealing the underlying data?

This is the central objective of *Secure Multi-Party Computation* (MPC).

2 Motivation: Secure Computation in ML Services

Consider a Machine Learning as a Service (MLaaS) setting. A model owner trains a machine learning model and deploys it on a cloud server. Users send queries to the server and receive predictions.

There are several pieces of potentially sensitive information in such a system. The model owner may want to protect the model itself, while a user may want to protect the query or data submitted to the model. The cloud server may also be able to observe the queries, intermediate computations, or model information.

Therefore, simply specifying the functionality of the system is not sufficient. We must also specify which entities are trusted and what an adversary is allowed to do.

3 Adversarial Model

An adversarial model specifies the assumptions we make about the behavior and capabilities of potentially compromised entities in a system. In particular, it specifies which entities may be compromised, how many may be compromised, how they may behave, and what information or control they have.

The adversarial model is not a property of the entire system by itself. Different entities in the same system may be assigned different trust assumptions.

3.1 Trusted Entity

A trusted entity is assumed to follow the specified protocol and is not considered to be compromised. Consequently, the protocol does not need to protect against arbitrary deviation by that entity.

For example, a model owner may be assumed to be trusted with respect to the generation and deployment of the model. This assumption does not necessarily mean that the model must be publicly revealed.

3.2 Semi-Honest Entity

A semi-honest entity, also called an honest-but-curious entity, follows the protocol correctly but attempts to learn additional information from everything available to it during the execution.

Such an entity does not modify messages or intentionally produce an incorrect result. Its goal is to extract information beyond what it is supposed to learn.

For example, a semi-honest cloud server may correctly execute an inference protocol but attempt to infer information about a user's private query from the messages and intermediate values it observes.

3.3 Malicious Entity

A malicious entity may arbitrarily deviate from the protocol. It may provide incorrect inputs, modify or omit messages, send inconsistent information to different entities, or deliberately manipulate the computation.

Thus, security against malicious entities generally requires protection of both privacy and correctness.

3.4 Effect of the Adversarial Model

The security requirements of a system depend directly on its adversarial model. A protocol designed only for semi-honest entities may not remain secure when an entity is allowed to behave maliciously.

For example, a cloud server may be assumed to be trusted, semi-honest, or malicious. In the trusted setting, the server may directly access the information required for computation. In the semi-honest setting, cryptographic techniques may be required to prevent the server from learning sensitive information. In the malicious setting, the protocol must additionally ensure that the server cannot arbitrarily manipulate the computation without being detected or prevented.

Therefore, there is no meaningful notion of a “secure system” without specifying the corresponding threat model.

4 System Model and Security Design

Designing a secure cryptographic system generally begins with defining the system model.

First, we identify the entities participating in the system and specify their relationships. We determine what information each entity possesses, what communication channels exist, and what computation is to be performed.

Next, we define the adversarial model. We specify which entities may be compromised and whether they are assumed to be trusted, semi-honest, or malicious. We may also need to specify the number of compromised entities and their capabilities.

Finally, we define the security goals. These may include confidentiality, correctness, integrity, authentication, privacy, or availability.

Only after these aspects are specified can we select appropriate cryptographic mechanisms.

The choice of primitive therefore follows the security model rather than preceding it. Depending on the requirements, the system may use encryption, commitments, zero-knowledge proofs, secret sharing, secure multi-party computation, oblivious transfer, homomorphic encryption, or combinations of these techniques.

5 Secure Multi-Party Computation

Secure Multi-Party Computation (MPC) is a cryptographic framework that enables multiple parties to jointly compute a function over their private inputs while protecting those inputs from the other parties.

Suppose there are n parties $P_1, \dots, P_n$, where P_i holds a private input s_i . The parties want to jointly compute a function $f(s_1, s_2, \dots, s_n)$.

The goal is for the parties to obtain the required output without revealing their private inputs beyond what is implied by the output and the specified security model.

The central principle is therefore:

Parties should be able to compute together without revealing their underlying private inputs.

The exact security guarantees depend on the adversarial model. An MPC protocol secure against semi-honest parties may guarantee privacy when all parties follow the protocol, whereas a protocol secure against malicious parties must additionally protect the correctness of the computation against arbitrary deviations.

6 A Simple Example of MPC

Consider several hospitals that want to determine the total number of patients satisfying a particular medical condition.

Each hospital has its own private patient database. Revealing the complete databases to the other hospitals would violate patient privacy. However, the hospitals may still want to obtain the aggregate number.

MPC allows the hospitals to jointly compute the desired aggregate while keeping their individual databases private.

The important point is that the computation is performed on protected representations of the inputs rather than by simply collecting all private databases at a single location.

7 MPC in Machine Learning Services

MPC can also be used in privacy-preserving machine learning services.

Suppose a model owner has a private model and a user has a private query. A secure computation protocol can allow the required inference to be performed without directly exposing the model or the query to the computing parties.

The precise protection depends on the system architecture and adversarial model. For example, the cloud server may be assumed to be semi-honest and may therefore execute the protocol correctly while attempting to infer information from its view. A stronger model may allow the server to behave maliciously, requiring additional mechanisms to ensure correctness.

This illustrates why secure computation is closely connected to the adversarial model: changing the assumed behavior of an entity changes the security mechanisms required by the protocol.

8 Post-Quantum Considerations

If the system must remain secure against quantum adversaries, the cryptographic primitives used in the construction must also provide appropriate post-quantum security.

This introduces additional challenges. Post-quantum primitives may have larger keys, ciphertexts, signatures, or communication overheads than their classical counterparts. Secure computation itself can also involve substantial computation and communication.

Thus, an important research challenge is to construct secure computation protocols that provide strong security guarantees against quantum adversaries while keeping computation and communication practical.

9 From Cryptographic Primitives to Secure Computation

Bit commitment and zero-knowledge proofs provide mechanisms for hiding information and proving properties about hidden information. Secure multi-party computation extends this idea to an entire computation.

The progression can therefore be viewed as follows:

1. Define the system and identify its participating entities.
2. Specify the trust assumptions and adversarial model.
3. Define the required security properties.
4. Select suitable cryptographic primitives.
5. Construct a protocol that achieves the required functionality and security.

Secure computation provides a general framework in which multiple parties can jointly evaluate a function while protecting their private inputs. The next step is to understand how such protocols can actually be constructed, beginning with secret sharing and the computation of functions over shared secrets.